

Security

Garantire l'integrità dei dati creando "oggetti sicuri"

L'obiettivo di questa prima sezione è capire come costruire un oggetto che sia *sicuro*, cioè un oggetto di cui nessuno possa alterare i dati in modo indesiderato. Si parte da due proprietà fondamentali: **accessibilità** ed **estensibilità**.

Accessibilità e il principio del minimo privilegio

Definizione — Principio del minimo privilegio (least privilege): ogni componente di un programma deve avere il minor livello di accesso possibile, sufficiente solo a svolgere il proprio compito, e niente di più.

In pratica: variabili di istanza e metodi vanno marcati **private**, a meno che tu non abbia una ragione concreta per aumentarne la visibilità. Meno "porte aperte" ci sono, meno superfici di attacco esistono.

```
public class Account {  
    private double balance;    // private: nessuno lo tocca da fuori  
    private void updateBalance() { /* logica interna */ }  
}
```

Estensibilità e classi **final**

Definizione — Estensibilità: la possibilità che una classe venga estesa (sottoclassata) da altre classi.

Il problema di sicurezza è questo: se una classe può essere sottoclassata, i suoi metodi possono essere **sovrascritti** (override), e un malintenzionato può iniettare comportamento dannoso ridefinendo un metodo. La difesa è marcare la classe come **final**, così impedisce le sottoclassi.

```
public final class Department { ... } // non sottoclassabile
```

Oggetti immutabili

Definizione — Oggetto immutabile: un oggetto il cui stato non può essere modificato dopo la creazione. È intrinsecamente sicuro perché, non potendo cambiare, nessuno può corromperne i dati.

Le linee guida per creare un oggetto immutabile (tutte citate nella lezione) sono:

1. **Nessun metodo set** — se non ci sono setter, i dati non si possono cambiare dall'esterno.
2. **Tutti i campi private e final** — private per l'accesso, final perché ricevono un valore una sola volta (all'inizializzazione) e non possono più essere riassegnati.
3. **Impedire le sottoclassi** — classe **final**, così nessuno può sovrascrivere i metodi.
4. **(Tecnica alternativa) costruttore privato + factory method statico** — invece di un costruttore pubblico, rendi privato il costruttore e fornisci un metodo statico pubblico "di fabbrica" (factory) che lo richiama internamente. Questo è un secondo modo per bloccare la

sottoclassabilità: una sottoclasse non può invocare un costruttore privato della classe genitore, quindi non può nemmeno essere creata.

Nel corso viene mostrato un esempio con una classe `Department` resa `final`, con costruttore privato e un metodo statico `createNewInstance(...)` che restituisce un nuovo `Department`. La lezione sottolinea la "doppia protezione": la classe è `final` e il costruttore è `private`.

Campi mutabili vs immutabili — il vero tranello

Questo è il punto più tecnico e importante della sezione. Rendere i campi `private final non basta` se un campo è un riferimento a un oggetto *mutabile*.

Distinzione chiave:

- **Tipi primitivi** (`int`, `double`, ecc.): passati **per valore**. Quando li passi o li restituisci, viene copiato il valore. Nessun rischio.
- **Tipi immutabili** (es. `String`): sicuri automaticamente. Se qualcuno prova a "modificarli", in realtà Java crea un nuovo oggetto in background e l'originale resta intatto.
- **Tipi mutabili** (es. `ArrayList`, `StringBuilder`): pericolosi. Qui devi intervenire.

Perché sono pericolosi: in Java il passaggio è sempre *by value*, ma per gli oggetti il "valore" che viene copiato è **il riferimento** (l'indirizzo). Quindi se il tuo costruttore memorizza direttamente il riferimento a un `ArrayList` ricevuto dall'esterno, chi ti ha passato quella lista continua a puntare **allo stesso oggetto interno**. Può modificarlo dopo, scavalcando la tua immutabilità. Lo stesso vale se un getter restituisce direttamente il riferimento interno.

Nel corso viene mostrato l'**esempio dei dipendenti (employees)**:

- Si crea una `ArrayList` con alcuni dipendenti (nell'audio, "e" e "Liam").
- Si costruisce un `Department` passando quella lista.
- Da un'altra classe (`Main`) si prova a cambiare lo stato dell'oggetto immutabile.
- I campi `String` (nome → provano a metterlo "Boots", indirizzo → "Galway") **non cambiano**, perché le stringhe sono immutabili.
- Ma quando si aggiunge un nuovo dipendente ("Tom") alla lista, **Tom compare dentro il dipartimento**. L'incapsulamento è rotto proprio sul campo mutabile.

La soluzione: copia difensiva (defensive copy)

Definizione — Copia difensiva: invece di memorizzare o restituire il riferimento originale a un oggetto mutabile, crei un **nuovo** oggetto con gli stessi dati. Così controlli tu il tuo oggetto interno e nessun riferimento esterno lo tocca.

Va applicata in **due punti**:

1. **In ingresso** (nel costruttore): quando ricevi un `ArrayList`, ne crei uno nuovo copiando i dati, invece di salvare il riferimento passato.
2. **In uscita** (nei getter): quando restituisci la lista, restituisci una copia nuova, non il riferimento interno.

```
public final class Department {
```

```

private final List<String> employees;

private Department(List<String> employees) {
    // copia difensiva IN INGRESSO
    this.employees = new ArrayList<>(employees);
}

public static Department createNewInstance(List<String> employees) {
    return new Department(employees);
}

public List<String> getEmployees() {
    // copia difensiva IN USCITA
    return new ArrayList<>(this.employees);
}
}

```

Con questa versione, quando dal Main si aggiunge "Tom" alla lista ottenuta dal getter, si modifica solo *quella copia*: l'ArrayList interno alla classe immutabile resta intatto. La lezione chiama questo approccio anche "**incapsulamento avanzato**" e precisa che copia difensiva e incapsulamento avanzato sono la stessa idea.

Riassunto della regola pratica: **primitivi e tipi immutabili** → **nessuna precauzione**; **tipi mutabili (ArrayList, StringBuilder)** → **copia difensiva in ingresso e in uscita**.

Attacchi di iniezione (Injection Attacks)

Definizione generale — Injection: un attacco in cui un input fornito dall'utente produce risultati inattesi perché viene interpretato come parte di un comando o di una query anziché come semplice dato.

La lezione ne distingue due tipi: **SQL injection** e **Command injection**.

SQL Injection

Definizione: l'input dell'utente viene concatenato dentro una query SQL e ne altera la logica, restituendo dati che non dovrebbero essere accessibili.

Esempio del corso (tabella conti bancari): c'è una tabella con codice filiale, numero di conto e dettagli cliente (incluso il saldo). Con una normale Statement che concatena l'input, un attaccante inserisce come "codice filiale" qualcosa tipo:

```
123456 OR branch_code IS NOT NULL
```

Il risultato è che la condizione diventa sempre vera e la query restituisce **tutte** le filiali, tutti i conti, tutti i clienti e tutti i saldi. Esattamente ciò che non vuoi.

Difesa — PreparedStatement con bind variables:

Definizione — Bind variable (parametro): un segnaposto (?) nella query SQL a cui il valore viene associato separatamente. Il valore viene trattato *sempre e solo come dato*, mai come codice SQL, quindi non può alterare la struttura della query.

```
// Vulnerabile (Statement con concatenazione)
```

```
String sql = "SELECT * FROM bank WHERE branch_code = " + input;
```

```
// Sicuro (PreparedStatement con bind variable)
```

```
String sql = "SELECT * FROM bank WHERE branch_code = ?";
```

```
PreparedStatement ps = conn.prepareStatement(sql);
```

```
ps.setString(1, input); // input trattato come semplice valore
```

Con la versione preparata, se passi un codice valido ("123456") ottieni la riga giusta; se passi la stringa malevola, questa viene *confrontata come testo* con la colonna `branch_code`, non corrisponde a nulla e **non ti viene restituito niente**. L'attacco fallisce.

Command Injection

Definizione: comandi del sistema operativo (o percorsi del filesystem) vengono manipolati tramite l'input utente per ottenere accessi o risultati non previsti.

Esempio del corso (la fattoria / farm): esiste una struttura di directory:

```
theFarm/
├── animals/
│   ├── cattle/ (Angus.txt, Hereford.txt) ← pubblici
│   └── sheep/ (Suffolk.txt) ← pubblico
└── staff/ ← RISERVATO (es. file che rivela che "Joe Bloggs" è stato disciplinato)
```

Il programma chiede all'utente di digitare una directory, costruisce un percorso con `Paths.get("farmAnimals/" + inputUtente)`, apre uno stream sui file e stampa i `.txt`. Se l'utente digita `cattle`, ottiene correttamente `Angus.txt` e `Hereford.txt`.

Il problema: se l'utente digita `..` (dot dot, "cartella superiore"), il percorso risale alla radice `theFarm` e lo stream elenca **tutte** le cartelle, incluse quelle riservate come `staff`. L'attaccante scopre così informazioni confidenziali già dai nomi dei file.

Difese — due tecniche, combinabili:

a) Whitelist (validazione dell'input)

Definizione — Whitelist: un elenco esplicito di ciò che è *consentito*. L'opposto è la **blacklist**, che elenca ciò che *non è consentito*.

Il codice controlla: la directory richiesta è `cattle` o `sheep`? Se sì, procede; se no, stampa "directory non valida". Digitando `..` ottieni ora il messaggio di errore, che è il comportamento desiderato.

La lezione sottolinea un vantaggio pratico: la whitelist va aggiornata quando aggiungi cartelle lecite (es. aggiungere `horses`), ma è comunque **più facile e più sicura** di una blacklist. Con la blacklist devi indovinare in anticipo tutte le cose "cattive" digitabili e restare sempre un passo avanti agli attaccanti — un gioco perdente.

b) Security policy (criteri di sicurezza)

Definizione — Security policy: un file di configurazione che concede permessi specifici (lettura, scrittura, ecc.) su risorse specifiche, applicando il principio del minimo privilegio.

Esempio: concedere solo permesso di *lettura* sul file `job_logs`; oppure lettura e scrittura sullo stesso file. La regola: se il programma deve solo leggere, dagli solo `read`, non `write`.

Definizione — Difesa in profondità (defense in depth): applicare più livelli di sicurezza sovrapposti (es. whitelist + security policy). Se un attaccante supera un livello, ne trova subito un altro. La combinazione di validazione dell'input e minimo privilegio è l'esempio citato di difesa in profondità.

Attacchi di negazione del servizio (Denial of Service, DoS)

Definizione — DoS: un attacco in cui una o più richieste vengono fatte con l'intento di interrompere il servizio per tutti gli altri utenti, tipicamente esaurendo le risorse del sistema.

La lezione elenca i modi principali in cui un programma Java può essere vulnerabile:

Perdita di risorse (resource leak)

Se un metodo apre un file ma non lo chiude, un attaccante chiama ripetutamente quel metodo finché la memoria (o gli handle di file) si esaurisce.

Soluzione: usare **sempre** `try-with-resources`, che chiude automaticamente le risorse.

```
try (BufferedReader br = Files.newBufferedReader(path)) {  
    // uso del file  
} // chiusura automatica garantita, niente leak
```

File estremamente grandi

Ricevere un file, elaborarlo e produrne uno nuovo va bene per file piccoli, ma un file enorme può esaurire la memoria.

Zip bomb

Definizione — Zip bomb: un file compresso che al suo interno contiene molti altri file (o dati che si espandono enormemente in decompressione), progettato per saturare la memoria/disco quando viene estratto. Va gestito con attenzione.

Strutture dati molto grandi

Ad esempio un array di dimensioni abnormi. In genere occorre **validare la dimensione** dell'array (controllare che rientri in limiti ragionevoli) prima di allocarlo o processarlo.

Informazioni riservate (Confidential Information)

Definizione — Informazioni riservate: dati privati e personali come password, indirizzo, data di nascita, stipendio, saldo del conto. Vanno protetti in memoria e nell'output.

Linee guida della lezione:

Mai loggare dati sensibili

I dati sensibili non devono mai finire a schermo o in una **stack trace** (la traccia dello stack stampata quando avviene un'eccezione). Un log o un crash potrebbero esporli.

Attenzione ai memory dump

Definizione — Memory dump (file di dump): un file generato quando un programma si blocca, che contiene un'istantanea della memoria. Se lì dentro ci sono dati riservati, un attaccante può leggerli.

char[] invece di String per le password

Questo è il punto tecnico centrale della sezione. La `System.console().readPassword()` restituisce un **array di caratteri (char[])** e non una `String`.

Perché: le `String` in Java sono immutabili e possono finire nello **string pool**, dove restano in memoria anche dopo che il codice che le usava ha terminato — non le controlli tu, non sai quando il garbage collector le rimuove. Un `char[]`, invece, puoi **sovrascriverlo o azzerarlo** manualmente appena hai finito, rimuovendo subito il dato sensibile dalla memoria.

```
char[] password = System.console().readPassword();  
  
// ... uso ...  
  
Arrays.fill(password, ' '); // cancello subito il dato sensibile
```

Impostare a null i riferimenti riservati

Appena hai finito di usare un oggetto che contiene dati riservati, imposta il suo riferimento a `null`. Questo lo rende immediatamente **idoneo alla garbage collection**, così il dato sparisce dalla memoria prima.

Principio guida finale: i dati riservati devono restare in memoria **per il minor tempo possibile**. Meno tempo restano, meno finestra ha l'attaccante per rubarli.